DeliverIQ

Food Delivery Operations & Customer Analytics

Complete Data Science Project Pipeline

From Raw Data to Production-Ready Dashboard

15,000	30	8	3
Synthetic Records	Features	Pipeline Phases	ML Model Families

Python 3.11+	Pandas / NumPy	Scikit-learn	XGBoost / LightGBM	Streamlit
Plotly	GitHub Actions	Docker	MLflow	SHAP

GitHub Repository: deliveriq-analytics

Report: DeliverIQ — Food Delivery Intelligence Pipeline

Designed for learning · Exploratory Analysis · ML Modeling · Streamlit Dashboard · CI/CD

Table of Contents

1 · Project Identity

Report Name

DeliverIQ — Food Delivery Intelligence Pipeline

GitHub Repository Name

`deliveriq-analytics`

GitHub Repository Description

End-to-end data science project on 15K food-delivery records: automated cleaning pipeline, EDA dashboards, multi-model ML (regression, classification, clustering), SHAP explainability, and an interactive Streamlit analytics app deployed via Docker and GitHub Actions CI/CD.

GitHub Topics / Tags

data-science	machine-learning	food-delivery	streamlit	python	eda	xgboost
shap	github-actions	docker	mlflow	pandas	scikit-learn	analytics-dashboard

2 · Dataset Column Dictionary — All 30 Features

The dataset contains 15,000 synthetic records and 30 numerical features designed to mirror real-world food-delivery platform dynamics. Each column is described below with its data type, business meaning, and primary role in analysis or modelling.

Column	Type	Description	Analytics Role
`order_id`	*Integer / ID*	Unique identifier for each delivery order. No analytical value; used only for joins and deduplication.	Primary Key
`city_tier`	*Ordinal (1–3)*	Market tier of the city: Tier-1 = metro, Tier-2 = mid-size, Tier-3 = small city. Influences demand, traffic, and operations.	Segmentation / Feature
`order_month`	*Integer (1–12)*	Calendar month the order was placed. Captures seasonality, festive spikes, and weather patterns.	Temporal Feature
`order_day_of_week`	*Integer (0–6)*	Day of the week (0=Monday … 6=Sunday). Captures weekly demand rhythms and weekend peaks.	Temporal Feature
`order_hour`	*Integer (0–23)*	Hour of the day the order was placed. Identifies lunch / dinner peaks and late-night orders.	Temporal Feature
`customer_age`	*Float*	Age of the customer in years. Used to segment users and understand age-based ordering behavior and preferences.	Customer Feature
`customer_loyalty_score`	*Float (0–100)*	Platform loyalty score derived from order history, repeat purchases, and engagement. Higher = more loyal customer.	Customer Feature / Target
`premium_customer_flag`	*Binary (0/1)*	Indicates whether the customer holds a premium subscription. Premium users may receive faster delivery, lower fees, or special promos.	Segmentation Flag
`order_value`	*Float (USD)*	Gross monetary value of the food items ordered, before discounts, fees, and tips.	Revenue Feature
`number_of_items`	*Integer*	Total number of individual items in the order. Influences preparation time and packaging complexity.	Order Feature
`discount_amount`	*Float (USD)*	Total monetary discount applied to the order via coupons, loyalty rewards, or platform offers.	Revenue Feature
`delivery_fee`	*Float (USD)*	Fee charged to the customer for the delivery service. May vary with distance, demand, and membership tier.	Revenue Feature
`tip_amount`	*Float (USD)*	Tip given by the customer to the delivery partner. Indicator of customer satisfaction and partner earnings.	Revenue / Satisfaction
`final_amount_paid`	*Float (USD)*	Total amount paid: order_value – discount_amount + delivery_fee + tip_amount. The actual revenue captured.	Target / Revenue

Column	Type	Description	Analytics Role
`promo_code_used`	*Binary (0/1)*	Whether a promotional code was applied. Useful for analyzing discount efficiency and promo ROI.	Marketing Flag
`delivery_distance_km`	*Float (km)*	Physical distance between the restaurant and the customer's delivery address in kilometres.	Logistics Feature
`preparation_time_minutes`	*Float (min)*	Time taken by the restaurant to prepare the order after it is placed.	Operational Feature
`delivery_time_minutes`	*Float (min)*	Actual end-to-end delivery time from order placement to doorstep drop-off.	Operational Feature / Target
`estimated_delivery_time`	*Float (min)*	Platform's predicted delivery time shown to the customer at checkout. Compare with actual for accuracy analysis.	Prediction Benchmark
`delayed_delivery_flag`	*Binary (0/1)*	1 if actual delivery exceeded estimated time by a defined threshold. Key KPI for SLA compliance.	Operational Target
`delivery_efficiency_score`	*Float (0–100)*	Composite score measuring partner route efficiency, speed, and order accuracy. Derived metric for partner evaluation.	Performance KPI
`traffic_level_score`	*Float (0–10)*	Encoded traffic congestion level at the time of delivery. Higher values indicate heavier traffic, increasing delivery time.	External Feature
`weather_severity_score`	*Float (0–10)*	Severity of weather conditions (rain, storm, etc.) at delivery time. Impacts delivery safety and timing.	External Feature
`festival_or_weekend_flag`	*Binary (0/1)*	1 if the order was placed on a weekend or during a local festival. Associated with demand spikes and delays.	Context Flag
`restaurant_rating`	*Float (1–5)*	Customer rating given to the restaurant for food quality. Influences restaurant ranking on the platform.	Quality KPI
`delivery_partner_rating`	*Float (1–5)*	Customer rating given specifically to the delivery partner for service quality and punctuality.	Quality KPI
`customer_rating`	*Float (1–5)*	Overall platform experience rating given by the customer combining food, delivery, and app experience.	Satisfaction Target
`delivery_partner_experience_years`	*Float*	Years of experience of the delivery partner. Experienced partners typically navigate better and have higher efficiency scores.	Partner Feature
`cancellation_flag`	*Binary (0/1)*	1 if the order was cancelled before delivery. Can be customer or partner-initiated. Key churn signal.	Operational Target
`refund_flag`	*Binary (0/1)*	1 if the order resulted in a refund. Often tied to missing items, wrong orders, or extreme delays.	Financial Risk Flag

> ***Note on Missing Values:*** *The dataset intentionally includes slight missingness (~1–3%) in numeric columns to simulate realistic data quality challenges. Handle these with median imputation (numerical) or mode imputation (categorical flags) during Phase 2.*

3 · Project Folder Structure

The following structure follows a production-grade data science layout suitable for GitHub, CI/CD integration, Docker containerisation, and Streamlit deployment.

```
deliveriq-analytics/
├── .github/workflows/
│   ├── ci.yml              ← Lint, test & coverage on every push/PR
│   ├── cd.yml              ← Build & push Docker on merge to main
│   └── data_validation.yml ← Great Expectations quality gate
├── data/
│   ├── raw/food_delivery.csv         ← Original CSV (never modified)
│   ├── processed/food_delivery_clean.csv
│   └── external/                     ← Supplementary data (holidays…)
├── notebooks/
│   ├── 01_data_audit.ipynb
│   ├── 02_eda.ipynb
│   ├── 03_feature_engineering.ipynb
│   ├── 04_modeling_regression.ipynb
│   ├── 05_modeling_classification.ipynb
│   ├── 06_modeling_clustering.ipynb
│   └── 07_shap_explainability.ipynb
```

```
├── src/
│   ├── config.py          ← Paths, constants, seeds
│   ├── data/loader.py · cleaner.py · validator.py
│   ├── features/engineer.py · selector.py
│   ├── models/train.py · evaluate.py · predict.py
│   └── viz/plots.py       ← Reusable Plotly helpers
├── app/
│   ├── main.py            ← Streamlit entry point
│   ├── pages/
│   │   ├── 1_Overview.py · 2_Delivery.py · 3_Customer.py
│   │   └── 4_Revenue.py · 5_ML_Insights.py · 6_Operations.py
│   └── components/kpi_card.py · filters.py
├── models/                ← Serialised .pkl artifacts
├── mlruns/                ← MLflow experiment tracking
├── tests/test_cleaner.py · test_features.py · test_models.py
├── reports/figures/       ← Exported charts
├── Dockerfile · docker-compose.yml
├── Makefile · pyproject.toml
├── requirements.txt · requirements-dev.txt
└── README.md
```

Key File Purposes

Makefile
Single-command shortcuts: make clean, make train, make app, make test, make lint.

pyproject.toml
Centralises tool config (Ruff, Black, pytest). Avoids sprawling config files.

.github/workflows/
Three workflow files: CI (quality gate), CD (Docker push), data validation.

src/ package
All reusable Python logic. Notebooks import from here — no copy-paste code.

app/pages/ Each Streamlit page is a separate file. Navigation is automatic via file names.	**mlruns/** MLflow auto-created folder. Track every experiment, params, metrics, and artifacts.
models/*.pkl Serialised final models. Loaded by the Streamlit app for live inference.	**tests/** Pytest unit tests. Run automatically on every PR via GitHub Actions.

PHASE 1 — Data Ingestion & Audit

Before any modelling or cleaning, deeply understand the shape, quality, and distributions of the raw dataset. This phase produces a data quality report and defines your cleaning agenda.

Goals

- Load raw CSV with Pandas; inspect shape, dtypes, memory usage.
- Profile each column: min, max, mean, median, std, skewness, kurtosis.
- Identify and quantify missing values (isnull().sum(), missingo heatmap).
- Spot duplicate order_id rows and impossible value ranges.
- Generate an automated profiling report using ydata-profiling (ProfileReport).
- Document all findings in notebooks/01_data_audit.ipynb.

Tools & Libraries

pandas Core data loading and manipulation.	**ydata-profiling** Auto-generates an HTML report with distributions and correlations.
missingno Visual matrix/heatmap of missing value patterns.	**great_expectations** Define schema expectations and run data validation suite.

Key Code Snippet

```
import pandas as pd
from ydata_profiling import ProfileReport

df = pd.read_csv('data/raw/food_delivery.csv')
print(df.shape, df.dtypes)
print(df.isnull().sum().sort_values(ascending=False))

profile = ProfileReport(df, title='DeliverIQ Audit', explorative=True)
profile.to_file('reports/data_audit_report.html')
```

PHASE 2 — Cleaning & Preprocessing

Transform raw data into a clean, ML-ready dataset. All cleaning logic lives in src/data/cleaner.py so it can be unit-tested and reused by the Streamlit app.

Step-by-Step Cleaning Agenda

1. **Drop Duplicates** — df.drop_duplicates(subset='order_id', inplace=True)
2. **Handle Missing Values** — Numeric columns: fill with median. Binary flags (0/1): fill with mode. Rows missing >30% features: consider dropping entirely.

3. Outlier Detection — Use IQR fencing (Q1 - 1.5×IQR, Q3 + 1.5×IQR) for continuous columns like delivery_time_minutes, order_value, tip_amount. Cap (clip) rather than drop.

4. Range Validation — Enforce domain constraints: ratings in [1, 5], flags in {0, 1}, scores in [0, 100], hours in [0, 23]. Flag rows that violate constraints.

5. Type Casting — Ensure integer columns (order_hour, city_tier, etc.) are int64, not float64 (common after imputation).

6. Encode Ordinal Features — city_tier is already numeric (1–3). If you add categorical text columns later, use OrdinalEncoder or LabelEncoder.

7. Feature Scaling — StandardScaler for linear models (Logistic Regression, Linear Regression). No scaling needed for tree-based models (XGBoost, Random Forest).

8. Save Clean Dataset — Export to data/processed/food_delivery_clean.csv with a version timestamp.

PHASE 3 — Exploratory Data Analysis (EDA)

EDA is where you build business intuition from the data. Create both static (Matplotlib/Seaborn) and interactive (Plotly) charts that will later appear in the Streamlit dashboard.

EDA Checklist by Category

Univariate Analysis
Histograms and KDE plots for every continuous column. Bar charts for discrete/flag columns. Identify skewed distributions.

Bivariate Analysis
Scatter: delivery_distance_km vs delivery_time_minutes. Box plots: customer_rating by city_tier and premium_customer_flag. Violin: delivery_time_minutes by traffic_level_score bins.

Correlation Heatmap
Seaborn heatmap of Pearson correlation for all 30 features. Identify multicollinearity. Focus on correlations with target columns.

Time-Based Trends
Line chart: average order_value by order_hour. Heatmap: avg delivery_time by order_day_of_week × order_hour. Monthly revenue trend with festival_or_weekend_flag overlay.

Operational Insights
Delayed vs on-time delivery breakdown by city_tier and traffic_level. Cancellation rate by order_hour, city_tier, and weather_severity_score. Refund rate by restaurant_rating quartile.

Partner Analysis
Scatter: partner experience years vs delivery_efficiency_score. Box plot: tip_amount by delivery_partner_rating. Distribution of partner_rating grouped by delayed_delivery_flag.

Customer Segments
Scatter: customer_loyalty_score vs customer_age coloured by premium_customer_flag. Promo code usage rate by loyalty score decile. Average final_amount_paid by loyalty score quartile.

Revenue Breakdown
Stacked bar: revenue components (order_value, delivery_fee, tip) by city_tier. Discount impact: order_value vs final_amount_paid delta. Promo ROI: compare average order_value for promo vs non-promo orders.

PHASE 4 — Feature Engineering

Create new informative features that the raw columns cannot express alone. Good features often matter more than model choice.

Derived Features to Create

delivery_delay_gap delivery_time_minutes – estimated_delivery_time. Positive = late, negative = early.	**cost_per_km** delivery_fee / delivery_distance_km. Measures pricing efficiency per km.
discount_pct discount_amount / order_value × 100. Percentage discount applied.	**revenue_per_item** final_amount_paid / number_of_items. Average revenue per item.
is_peak_hour Binary: 1 if order_hour in {12,13,14,19,20,21}. Lunch and dinner rush.	**is_night_order** Binary: 1 if order_hour < 6 or order_hour > 22. Late-night segment.
prep_to_deliver_ratio preparation_time_minutes / delivery_time_minutes. Bottleneck identifier.	**stress_score** Composite of traffic_level_score × weather_severity_score. Environment load index.
total_experience_proxy delivery_partner_experience_years × delivery_efficiency_score. Partner quality score.	**high_value_order** Binary: 1 if order_value > 75th percentile. Premium order flag.

Feature Selection Methods

- **Filter:** Pearson correlation, mutual information scores, variance threshold.
- **Wrapper:** Recursive Feature Elimination (RFE) with Random Forest estimator.
- **Embedded:** XGBoost feature_importances_ and SHAP mean absolute values.
- **Collinearity:** VIF (Variance Inflation Factor) to remove redundant features.
- Track feature set versions in MLflow as params for reproducibility.

PHASE 5 — Machine Learning Models

Three ML families address different business questions in the dataset. All experiments are tracked with MLflow. Use scikit-learn Pipelines to prevent data leakage and simplify deployment.

REGRESSION — Predict delivery_time_minutes

- Baseline: Linear Regression, ElasticNet
- Ensemble: Random Forest Regressor, Gradient Boosting
- Advanced: XGBoost Regressor, LightGBM Regressor
- Metrics: MAE, RMSE, R^2, MAPE
- CV: 5-fold cross-validation + hold-out test set

CLASSIFICATION — Predict cancellation_flag / delayed_delivery_flag

- Baseline: Logistic Regression, Decision Tree
- Ensemble: Random Forest, Gradient Boosting Classifier
- Advanced: XGBoost Classifier, LightGBM Classifier
- Metrics: Accuracy, F1-score, ROC-AUC, Precision-Recall curve
- Handle class imbalance: SMOTE oversampling or class_weight='balanced'

CLUSTERING — Segment customers

- K-Means Clustering (Elbow method + Silhouette score for k selection)
- DBSCAN for noise-robust spatial clustering
- Features: customer_age, loyalty_score, order_value, frequency proxies
- Visualise with PCA (2D) or t-SNE plots
- Label clusters: 'Budget Casual', 'Loyal Premium', 'At-Risk', etc.

Hyperparameter Tuning

- **Optuna** — Bayesian optimisation for XGBoost/LightGBM. Efficient trial pruning.
- **GridSearchCV / RandomizedSearchCV** — For simpler models and quick baselines.
- **MLflow** — Log every trial: params, metrics, model artifact. Compare runs in UI.
- **Early Stopping** — Use with XGBoost eval_set to prevent overfitting automatically.

PHASE 6 — Model Explainability — SHAP

SHAP (SHapley Additive exPlanations) provides model-agnostic, mathematically grounded explanations for every prediction. Essential for building trust with business stakeholders and identifying operational levers.

SHAP Analysis Checklist

- **Global Importance:** shap.summary_plot() — Bar chart of mean |SHAP value| per feature.
- **Beeswarm Plot:** Shows direction + magnitude of each feature's impact across all samples.
- **Dependence Plot:** How SHAP value of delivery_distance_km changes with its actual value.
- **Waterfall Plot:** Explain a single prediction — e.g., why was this order delayed?
- **Force Plot:** Interactive HTML showing push/pull forces for individual predictions.
- **TreeExplainer:** Use for XGBoost / Random Forest (fast, exact values).
- **Business Insight:** Top 3 features driving cancellation. Top 3 features inflating delivery time.
- Export SHAP plots as PNG to reports/figures/. Embed in the Streamlit dashboard page.

PHASE 7 — Streamlit Dashboard

The Streamlit app transforms analysis into an interactive, business-facing tool. Six pages cover all key operational and analytical domains. Run with: streamlit run app/main.py

Page 1: Overview KPI scorecards (total orders, avg delivery time, cancellation rate, revenue). Trend line of daily orders. City-tier breakdown pie. Filter by date range.	**Page 2: Delivery Performance** Actual vs estimated delivery time scatter. Delay rate by hour/day heatmap. Distance vs time regression line. Traffic & weather impact bars.
Page 3: Customer Insights Loyalty score distribution. Scatter: age vs loyalty by premium flag. Promo usage funnel. Cluster assignment donut chart.	**Page 4: Revenue & Discounts** Revenue waterfall (order_value → discounts → fees → tips → final). Discount % vs order_value scatter. Promo ROI comparison.
Page 5: ML Predictions Live input form → predict delivery time or cancellation probability. SHAP waterfall for the prediction. Feature importance bar chart.	**Page 6: Operations** Cancellation heatmap. Refund analysis by restaurant_rating. Partner performance scatter (experience vs efficiency). City-tier SLA compliance.

Dashboard Technical Tips

- Use st.cache_data on data loading and model loading — critical for performance.
- Implement sidebar filters (city_tier, date range, customer type) that update all charts.
- Use plotly.express for interactive charts. Wrap in st.plotly_chart(use_container_width=True).
- Store loaded model in st.session_state to avoid reloading on every interaction.
- Add st.download_button to allow exporting filtered data as CSV.
- Deploy on Streamlit Community Cloud (free) or as a Docker container on any cloud.

PHASE 8 — GitHub Actions CI/CD

GitHub Actions automates quality checks and deployment, ensuring every code change is validated before reaching production.

Workflow 1: ci.yml (on push and pull_request)

```yaml
name: CI Pipeline
on: [push, pull_request]
jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.11'}
      - run: pip install -r requirements-dev.txt
      - run: ruff check src/ app/ tests/     # Linting
      - run: black --check src/ app/ tests/  # Formatting
      - run: pytest tests/ --cov=src --cov-report=xml
      - uses: codecov/codecov-action@v4      # Coverage badge
```

Workflow 2: cd.yml (on push to main)

```yaml
name: CD — Build & Deploy
on:
  push:
    branches: [main]
jobs:
  docker:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKER_USER }}
          password: ${{ secrets.DOCKER_TOKEN }}
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: yourdockerid/deliveriq:latest
```

Workflow 3: data_validation.yml (on schedule, weekly)

- Run Great Expectations data validation suite on the processed dataset.
- Fail the workflow (and send Slack/email alert) if data quality drops below threshold.
- Schedule with: on: schedule: - cron: '0 8 * * 1' (every Monday at 8 AM).

Dockerfile

```dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8501
CMD ["streamlit", "run", "app/main.py", "--server.port=8501", "--server.headless=true"]
```

12 · Full Project Report Structure

Structure your written report (PDF or Jupyter Book) with these sections. Each section maps to a pipeline phase and a set of deliverables.

Section	Length	Key Deliverables
0. Executive Summary	1 page	Problem statement, key findings, 3 business recommendations, dashboard link.
1. Introduction & Business Context	1–2 pages	Platform overview, dataset description, 30 columns explained, analytical objectives.
2. Data Audit & Quality Report	2–3 pages	Missing values heatmap, outlier boxplots, dtype table, ydata-profiling link.
3. Cleaning & Preprocessing	2 pages	Before/after statistics, imputation decisions, outlier treatment rationale.
4. Exploratory Data Analysis	5–7 pages	12–15 charts across all 8 EDA categories. Each chart has 2–3 sentence business insight.
5. Feature Engineering	2 pages	10 new feature definitions, feature importance ranking table, correlation delta before/after.
6. Regression Modelling	3 pages	Model comparison table (MAE, RMSE, R^2), learning curves, residual plots.
7. Classification Modelling	3 pages	Confusion matrices, ROC curves, PR curves, model leaderboard table.
8. Customer Clustering	2 pages	Elbow & silhouette plots, cluster profiles table, PCA 2D scatter plot.
9. Model Explainability	2–3 pages	SHAP summary, beeswarm, dependence and waterfall plots. Business narrative per model.
10. Streamlit Dashboard Guide	1–2 pages	Screenshot of each page, filter descriptions, how-to-use instructions.
11. GitHub Actions & DevOps	1 page	CI/CD workflow diagram, badge URLs, Docker deployment steps.
12. Business Recommendations	1–2 pages	3–5 data-driven recommendations with supporting evidence and expected impact.
13. Conclusions & Next Steps	1 page	Key learnings, model limitations, suggested future work (real-time data, A/B testing).
Appendix A	Optional	Raw profiling report, full feature correlation matrix, hyperparameter search results.
Appendix B	Optional	Code snippets for reproducibility. Link to GitHub repo and Streamlit deployment URL.

13 · Extra Recommendations & Advanced Suggestions

MLflow Experiment Tracking
Log every model run: mlflow.log_params(), mlflow.log_metrics(), mlflow.sklearn.log_model(). Run the MLflow UI with mlflow ui. Compare 20+ runs visually. Set up Model Registry to promote best models to Staging → Production.

DVC — Data Version Control
Track data files with DVC (like Git for data). dvc add data/raw/food_delivery.csv stores file hash in Git. Use dvc repro to replay the full pipeline. Connect to remote storage (S3, GCS) for team collaboration.

FastAPI Inference Endpoint
Wrap your best model in a FastAPI endpoint: POST /predict with JSON input returns predicted delivery_time_minutes and cancellation probability. Containerise with Docker. Your Streamlit app can call this API for real-time predictions.

Great Expectations — Data Contracts
Define expectations: delivery_time_minutes must be between 5 and 180. customer_rating must be between 1 and 5. city_tier must be in {1,2,3}. Run in CI pipeline to catch data drift or upstream schema changes.

Pre-commit Hooks
Install pre-commit with hooks: ruff (lint), black (format), isort (imports), nbstripout (clear notebook outputs). Prevents messy commits from ever entering the repo.

Jupyter Book / Quarto Report
Turn your Jupyter notebooks into a beautiful, navigable HTML/PDF report with Jupyter Book or Quarto. Auto-publish to GitHub Pages via GitHub Actions.

A/B Testing Simulation
Use the discount_amount and promo_code_used columns to simulate an A/B test: compare avg order_value for promo vs no-promo groups. Apply bootstrap confidence intervals. Calculate statistical significance with a two-sample t-test or Mann-Whitney U.

Time-Series Forecasting
Aggregate data to daily/hourly level and forecast order volume using Prophet or ARIMA. Useful for capacity planning. Use order_hour, order_day_of_week, festival_or_weekend_flag as regressors in Prophet for better seasonality capture.

Interactive EDA with Lux or D-Tale
pip install lux-api or dtale. One-line commands generate fully interactive, browser-based EDA UIs. Great for quick stakeholder demos and hypothesis generation.

Cookiecutter Data Science Template
For your next project, start with cookiecutter-data-science. Provides a standardised structure used across the industry. Run: cookiecutter https://github.com/drivendata/cookiecutter-data-science

Suggested Learning Resources

- **Hands-On ML with Scikit-Learn & TensorFlow** — Aurélien Géron. Deep coverage of pipelines and evaluation.
- **Python for Data Analysis** — Wes McKinney. Pandas mastery from the library's creator.
- **Streamlit Documentation** — docs.streamlit.io. Covers components, deployment, secrets management.
- **MLflow Documentation** — mlflow.org/docs. Tracking, projects, models, and registry.
- **SHAP Documentation** — shap.readthedocs.io. All explainer types and plot examples.
- **GitHub Actions Documentation** — docs.github.com/en/actions. Workflows, runners, secrets.
- **Kaggle Courses** — Free courses on Pandas, ML, Data Visualisation, and Feature Engineering.
- **DVC Documentation** — dvc.org/doc. Data versioning, pipelines, and experiment tracking.

Final Checklist Before Submission / Deployment

- ■ All notebooks run top-to-bottom without errors (use nbconvert --execute to verify).
- ■ README.md includes: project description, setup instructions, Streamlit URL, badge links.

- ■ GitHub Actions CI badge is green (passing) on the main branch.
- ■ All secrets (API keys, Docker credentials) are in GitHub Secrets — never hard-coded.
- ■ Streamlit dashboard tested on both desktop and mobile viewports.
- ■ MLflow model registry has the best model registered as 'Production'.
- ■ requirements.txt is pinned (==) for reproducibility. requirements-dev.txt for dev tools.
- ■ All SHAP plots saved to reports/figures/ and referenced in the written report.
- ■ Business recommendations section answers: 'What should the platform DO differently?'
- ■ Data and models are gitignored if large; tracked with DVC and stored remotely.

DeliverIQ — Food Delivery Intelligence Pipeline	GitHub: deliveriq-analytics